

Practical-1: Demo of Basic UI layout representing simple profile screen

Column, Row, Container, Padding, and Text widgets

A **customized** AppBar

Step 1: Create a Flutter Project

Open VS Code and Create New App—demo_business_card

Step 2: Replace lib/main.dart With This Code

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Profile App',
      theme: ThemeData(
        primarySwatch: Colors.teal,
      ),
      home: const ProfileScreen(),
    );
  }
}

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```

// Custom AppBar
appBar: AppBar(
  title: const Text(
    'User Profile',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
  centerTitle: true,
  backgroundColor: Colors.teal[800],
  elevation: 5,
  actions: const [
    Padding(
      padding: EdgeInsets.only(right: 12.0),
      child: Icon(Icons.settings),
    )
  ],
),

// 🏠 Body with Column, Row, Container, Text, Padding
body: Padding(
  padding: const EdgeInsets.all(16.0),
  child: Column(
    children: [

      // 🖼️ Profile Image Container
      Container(
        width: 120,
        height: 120,
        decoration: BoxDecoration(
          color: Colors.teal[100],
          shape: BoxShape.circle,
          image: const DecorationImage(
            image: AssetImage('assets/images/profile.jpg'), // Add image in assets
            fit: BoxFit.cover,
          ),
        ),
      ),

      const SizedBox(height: 16),
    ],
  ),
),

```

```
// 👤 Name Text
const Text(
  'Nimisha Patel',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 8),

const Text(
  'Computer Science Student | Flutter Enthusiast',
  style: TextStyle(fontSize: 16, color: Colors.grey),
),

const Divider(height: 30, thickness: 1.2),

// 📧 Info Row
Row(
  children: [
    const Icon(Icons.email, color: Colors.teal),
    const SizedBox(width: 10),
    const Text(
      'nimisha@example.com',
      style: TextStyle(fontSize: 16),
    ),
  ],
),

const SizedBox(height: 12),

Row(
  children: [
    const Icon(Icons.phone, color: Colors.teal),
    const SizedBox(width: 10),
    const Text(
      '+91 98765 43210',
      style: TextStyle(fontSize: 16),
    ),
  ],
),
```

```

    ],
  ),

  const Spacer(),

  // Bottom Button Row
  Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      ElevatedButton.icon(
        onPressed: () {},
        icon: const Icon(Icons.message),
        label: const Text('Message'),
      ),
      OutlinedButton.icon(
        onPressed: () {},
        icon: const Icon(Icons.call),
        label: const Text('Call'),
      ),
    ],
  ),
),
],
),
),
);
}
}

```

Step 3: Add an Image

Put your image in the assets/ folder.

Steps:

1. Create a folder named assets in the root directory.
2. Add your image file, e.g., profile.jpg.

Step 4: Update pubspec.yaml

Uncomment and update the assets section in your pubspec.yaml:

```
flutter:  
  assets:  
    - assets/profile.jpg
```

Then run:

flutter pub get

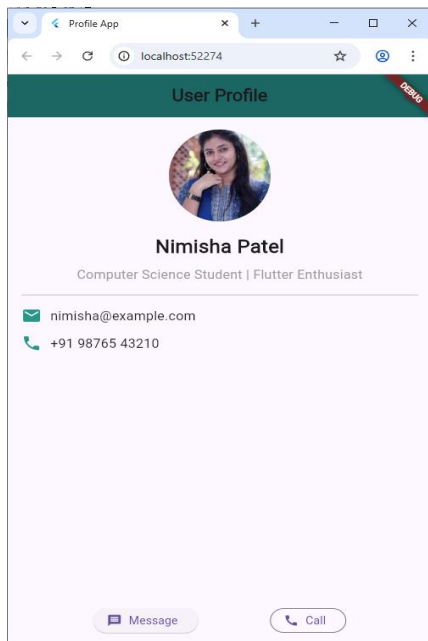
Step 5: Run the App

You can run the app with:

flutter run

Make sure an emulator or device/chrome browser is connected.

Output:



Practical – 2: demo_UI: Design a layout using Container, Column, Padding, Center (UI Layouts using Containers)

Step 1: Create a Flutter App

Open VS Code and Create New App—demo_container_layout_app

Step 2: Edit lib/main.dart With This Code

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Container Layout Demo',
      home: Scaffold(
        appBar: AppBar(
          title: Text('UI Layout with Containers'),
        ),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min, // Only use needed vertical space
            children: [
              Padding(
                padding: const EdgeInsets.all(8.0),
                child: Container(
                  width: 200,
                  height: 100,
                  color: Colors.blue,
                  child: Center(
                    child: Text(
                      'Container 1',
                      style: TextStyle(color: Colors.white, fontSize: 18),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

$$\begin{array}{c}) \\) \\) \\) \end{array}$$

```
Padding(  
padding: const EdgeInsets.all(8.0),  
child: Container(  
width: 200,  
height: 100,  
color: Colors.green,  
child: Center(  
child: Text(  
'Container 2',  
style: TextStyle(color: Colors.white, fontSize: 18),  
),  
),  
),  
),  
),
```

```
Padding(
  padding: const EdgeInsets.all(8.0),
  child: Container(
    width: 200,
    height: 100,
    color: Colors.orange,
    child: Center(
      child: Text(
        'Container 3',
        style: TextStyle(color: Colors.white, fontSize: 18),
      ),
    ),
  ),
),
),
),
],
),
),
),
),
);
}
```

```
}
```

Step 3: Run the App

You can run the app with:

flutter run

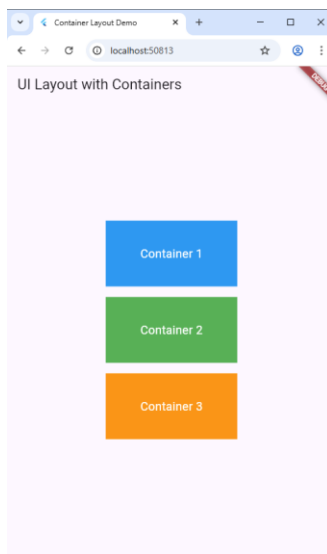
Make sure an emulator or device/chrome browser is connected.

You should see a centered column with **3 colorful containers**, each padded and containing centered text.

Explanation of Widgets Used

Widget	Purpose
Container	To hold visual content and give shape & color
Column	To arrange widgets vertically
Padding	To give space around each container
Center	To center the column in the body of the Scaffold

Output:



Practical-3: Write a program to Develop Area Calculator App using Flutter and VS Code

Step 1: Create a new Flutter project:

Open VS Code and Create New App—demo_area_calculator

Step 2: Modify lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const AreaCalculatorApp());

class AreaCalculatorApp extends StatelessWidget {
  const AreaCalculatorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Area Calculator',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.teal),
      home: const AreaCalculatorPage(),
    );
  }
}

class AreaCalculatorPage extends StatefulWidget {
  const AreaCalculatorPage({super.key});

  @override
  State<AreaCalculatorPage> createState() => _AreaCalculatorPageState();
}

class _AreaCalculatorPageState extends State<AreaCalculatorPage>
{
  final TextEditingController _lengthController = TextEditingController();
  final TextEditingController _widthController = TextEditingController();
  double? _area;
```

```

void _calculateArea() {
  final double? length = double.tryParse(_lengthController.text);
  final double? width = double.tryParse(_widthController.text);

  if (length != null && width != null) {
    setState(() {
      _area = length * width;
    });
  } else {
    setState(() {
      _area = null;
    });
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Please enter valid numbers')),
    );
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Area Calculator')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(

        children: [
          TextField(
            controller: _lengthController,
            decoration: const InputDecoration(labelText: 'Enter Length'),
            keyboardType: TextInputType.number,
          ),

          TextField(
            controller: _widthController,
            decoration: const InputDecoration(labelText: 'Enter Width'),
            keyboardType: TextInputType.number,

```

),

```
const SizedBox(height: 20),  
ElevatedButton(  
  onPressed: _calculateArea,  
  child: const Text('Calculate Area'),  
),
```

```
const SizedBox(height: 20),  
if (_area != null)  
  Text(  
    'Area: ${_area!.toStringAsFixed(2)} sq units',  
    style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
  ),  
],  
,  
,  
);  
}  
}
```

Step 3: Run the App

You can run the app with:

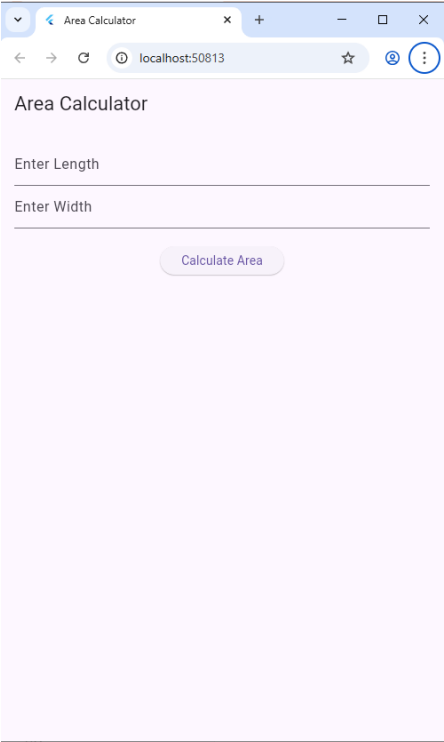
flutter run

Make sure an emulator or device/chrome browser is connected.

Output:

You will see a screen with:

- Two input fields for Length and Width
- A **"Calculate Area"** button
- Displays result: **"Area: _____ sq units"**



Practical-4: demo_area_calculator with dropdown & validations - changes in above app with:

- ✓ Dropdown for shape selection;
- ✓ Validation for negative numbers;
- ✓ Clear button

Step 1: Create a new Flutter project:

Open VS Code and Create New App—demo_area_calculator

Step 2: Modify lib/main.dart

```
import 'package:flutter/material.dart';
void main() => runApp(const AreaCalculatorApp());

class AreaCalculatorApp extends StatelessWidget {
  const AreaCalculatorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Area Calculator',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.teal),
      home: const AreaCalculatorPage(),
    );
  }
}

class AreaCalculatorPage extends StatefulWidget {
  const AreaCalculatorPage({super.key});

  @override
  State<AreaCalculatorPage> createState() => _AreaCalculatorPageState();
}

class _AreaCalculatorPageState extends State<AreaCalculatorPage>
{
  final TextEditingController _lengthController = TextEditingController();
  final TextEditingController _widthController = TextEditingController();
```

```
final TextEditingController _radiusController = TextEditingController();  
final TextEditingController _baseController = TextEditingController();  
final TextEditingController _heightController = TextEditingController();
```

```
String _selectedShape = 'Rectangle';  
double? _area;
```

```
final List<String> _shapes = ['Rectangle', 'Square', 'Circle', 'Triangle'];
```

```
void _calculateArea() {  
  try {  
    setState(() {  
      _area = null;  
    });
```

```
    switch (_selectedShape) {  
      case 'Rectangle':  
        double? l = double.tryParse(_lengthController.text);  
        double? w = double.tryParse(_widthController.text);  
        if (_isValid(l) && _isValid(w)) {  
          setState(() {  
            _area = l! * w!;  
          });  
        } else {  
          _showError();  
        }  
        break;
```

```
      case 'Square':  
        double? l = double.tryParse(_lengthController.text);  
        if (_isValid(l)) {  
          setState(() {  
            _area = l! * l;  
          });  
        } else {  
          _showError();  
        }  
        break;
```

```
      case 'Circle':
```

```

double? r = double.TryParse(_radiusController.text);
if (_isValid(r)) {
    setState(() {
        _area = 3.1416 * r! * r;
    });
} else {
    _showError();
}
break;

case 'Triangle':
    double? b = double.TryParse(_baseController.text);
    double? h = double.TryParse(_heightController.text);
    if (_isValid(b) && _isValid(h)) {
        setState(() {
            _area = 0.5 * b! * h!;
        });
    } else {
        _showError();
    }
    break;

}
}

catch (e) {
    _showError();
}
}

bool _isValid(double? val) => val != null && val >= 0;

void _showError() {
    ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Please enter valid positive numbers')),
    );
}

void _clearAll() {
    _lengthController.clear();
}

```

```
_widthController.clear();
_radiusController.clear();
_baseController.clear();
_heightController.clear();

setState(() {
  _area = null;
});
}
```

```
Widget _buildInputs() {
  switch (_selectedShape) {
    case 'Rectangle':
      return Column(
        children: [
          _buildNumberField(_lengthController, 'Length'),
          const SizedBox(height: 10),
          _buildNumberField(_widthController, 'Width'),
        ],
      );

    case 'Square':
      return _buildNumberField(_lengthController, 'Side Length');

    case 'Circle':
      return _buildNumberField(_radiusController, 'Radius');

    case 'Triangle':
      return Column(
        children: [
          _buildNumberField(_baseController, 'Base'),
          const SizedBox(height: 10),
          _buildNumberField(_heightController, 'Height'),
        ],
      );
  }
}
```

default:

```
    return const SizedBox();  
  }  
}
```

```
Widget _buildNumberField(TextEditingController controller, String label) {  
  return TextField(  
    controller: controller,  
    decoration: InputDecoration(  
      labelText: label,  
      border: const OutlineInputBorder(),  
    ),  
    keyboardType: TextInputType.number,  
  );  
}
```

@override

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('📐 Area Calculator'),  
      centerTitle: true,  
    ),  
    body: Padding(  
      padding: const EdgeInsets.all(16.0),  
      child: Column(  
        children: [  
          // Dropdown for shape selection  
          Row(  
            children: [  
              const Text(  
                'Select Shape: ',  
                style: TextStyle(fontSize: 16),  
              ),  
              const SizedBox(width: 10),  
  
              DropDownButton<String>(  
                value: _selectedShape,  
                items: _shapes.map((String shape) {
```

```

        return DropDownMenuItem<String>(
            value: shape,
            child: Text(shape),
        );
    }).toList(),
    onChanged: (String? newValue) {
        setState(() {
            _selectedShape = newValue!;
            _clearAll();
        });
    },
),
],
),

const SizedBox(height: 20),

// Dynamic input fields
_buildInputs(),

const SizedBox(height: 20),

// Buttons
Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
        ElevatedButton.icon(
            onPressed: _calculateArea,
            icon: const Icon(Icons.calculate),
            label: const Text('Calculate'),
        ),
        OutlinedButton.icon(
            onPressed: _clearAll,
            icon: const Icon(Icons.clear),
            label: const Text('Clear'),
        ),
    ],
),

const SizedBox(height: 30),

```

```

// Display Area
if (_area != null)
  Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.teal[100],
      borderRadius: BorderRadius.circular(10),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Icon(Icons.check_circle, color: Colors.teal),
        const SizedBox(width: 10),
        Text(
          'Area: ${_area!.toStringAsFixed(2)} sq units',
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
            color: Colors.teal,
          ),
        ),
      ],
    ),
  ),
];
};
}
}

```

Output:

The first screenshot shows the 'Area Calculator' interface with a dropdown menu open, listing 'Rectangle', 'Square', 'Circle', and 'Triangle'. The 'Rectangle' option is selected. The second screenshot shows the 'Rectangle' form with 'Length' set to 20 and 'Width' set to 30, resulting in an area of 600.00 sq units. The third screenshot shows the 'Square' form with 'Side Length' set to 50, resulting in an area of 2500.00 sq units.

Area Calculator

Select Shape: **Rectangle**

Base:

Height:

✓ Area: 600.00 sq units

Area Calculator

Select Shape: **Square**

Side Length:

✓ Area: 2500.00 sq units

The fourth screenshot shows the 'Area Calculator' interface with 'Circle' selected. The 'Radius' is set to 15, resulting in an area of 706.86 sq units. The fifth screenshot shows the 'Area Calculator' interface with 'Triangle' selected. The 'Base' is set to 5 and 'Height' is set to 10, resulting in an area of 25.00 sq units.

Area Calculator

Select Shape: **Circle**

Radius:

✓ Area: 706.86 sq units

Area Calculator

Select Shape: **Triangle**

Base:

Height:

✓ Area: 25.00 sq units